

BIG DATA ENGINEERING · INTERVIEW PREP

Apache Spark & PySpark

A complete interview-prep reference: Hadoop foundations, Spark's architecture and execution model, the full PySpark transformation toolkit, and a 50+ question interview bank.

Hadoop & HDFS

Spark Architecture

RDD & DataFrame

Catalyst Optimizer

PySpark

50+ Interview Q&A

PART ONE



Apache Spark Basics

Core concepts, architecture, and the execution model behind every Spark job — the foundation every other topic builds on.



1

What Is Big Data?

BIG DATA ENGINEERING · INTERVIEW PREP

Data that's too large, fast, or complex for traditional systems to store and process efficiently. Spark was built to handle it.



Volume

Huge amounts of data — often terabytes to petabytes.



Velocity

The speed at which data is generated and must be processed.



Variety

Different types of data — structured, semi-structured, unstructured.

HDFS — Distributed File System

- The storage layer of Hadoop
- Data is divided into blocks
- Default block size is 128 MB

MapReduce

- Programming model for processing large data
- The engine that processes very large datasets across multiple machines in a cluster

What Is Apache Spark?

- Apache Spark is up to 100x faster than MapReduce
- Spark is an alternative to Hadoop MapReduce — not to Hadoop itself
- Spark is a distributed computing engine that processes data in parallel
- It's a unified engine: one engine handles many types of data processing

PySpark

Python API

Spark SQL

SQL

MLlib

Machine Learning

GraphX

Graph processing

- **Cluster** — a group of interconnected machines
- **Node** — a single machine in the cluster
- **Cluster Manager** — manages cluster resources; decides which worker nodes have available CPU and memory
- **Driver** — the "brain" of Spark; controls the whole application
- **Worker Node** — a machine that performs the actual processing; runs executors
- **Executor** — a JVM process on a worker node that executes tasks, stores data in memory/disk, and returns results to the driver

What the Driver Controls

- Creates the SparkSession · builds the execution plan · divides work into tasks · sends tasks to executors · collects the results

SparkSession vs SparkContext

- **SparkContext** — the main connection between your program and the Spark cluster; mainly used for RDDs
- **SparkSession** — the entry point for DataFrames, Spark SQL, and reading CSV/JSON/Parquet
- **Key takeaway** — SparkSession contains a SparkContext internally — you rarely create SparkContext directly in modern Spark code

```
# SparkContext (legacy, RDDs)
from pyspark import SparkContext
sc = SparkContext(master="local[*]")

# SparkSession (modern, DataFrames)
from pyspark.sql import SparkSession
spark = SparkSession.builder \
    .appName("MySparkApp") \
    .master("local[*]") \
    .getOrCreate()
```

Narrow Transformation

- Each output partition depends on only one input partition
- No data movement / shuffle needed
- `map()`, `filter()`, `union()`, `flatMap()`, `mapPartitions()`

Wide Transformation

- Output partition depends on multiple input partitions
- Requires shuffling data between partitions
- `groupBy()`, `join()`, `distinct()`, `orderBy()`, `repartition()`

- Spark doesn't execute code until an action is called
- **Actions:** show(), collect(), count(), head(), first(), write.csv(), save()
- **Why it matters** — waiting lets Spark see the whole chain and optimize the plan, reducing unnecessary work

```
df_f = df.filter(df.salary > 4500)
df_s = df_f.select("name", "salary")
print("No execution yet!")

# Action -> triggers execution
df_s.show()
```

- The query optimization framework that makes SQL and DataFrame queries execute more efficiently
- Transforms your code/query through several stages to produce an optimized execution plan

Benefits

- Faster query execution
- Reduced CPU and memory usage
- Less data read from storage

Debugging

- **df.explain()** shows the plan · **df.explain(True)** shows parsed, analyzed, optimized, and physical plans

RDDs — Resilient Distributed Datasets

- The backbone of Apache Spark — its core data structure
- An immutable collection of data, with no schema
- **Resilient** — fault-tolerant; if a partition is lost, Spark recomputes it using lineage
- **Distributed** — data spread across many executors

Why DataFrame Instead of Raw RDD?

- RDDs have no schema, so Catalyst can't optimize them
- The DataFrame API lets Spark generate the most efficient code itself
- **Lineage** — the recorded history of transformations that lets Spark rebuild a lost RDD without replicating all the data

- Job — the highest execution unit; Spark creates one job every time an action is executed
- Stage — a job is divided into stages; a stage is a group of operations that run without a shuffle
- Task — the smallest execution unit; each task processes exactly one partition, on one executor core

Rule of thumb: Action → creates a Job · Shuffle → creates a new Stage · Partition → creates a Task

The command-line tool used to launch a Spark application on local machines, standalone clusters, YARN, Kubernetes, or Mesos.

```
spark-submit \  
  --master yarn \  
  --deploy-mode cluster \  
  --num-executors 10 \  
  --executor-cores 4 \  
  --executor-memory 8G \  
  --driver-memory 4G \  
word_count.py
```

```
# starts the driver -> creates the SparkSession -> connects to  
# the cluster -> requests executors -> sends tasks -> monitors  
# executors -> stops the application when finished
```

repartition()

- Used to increase or decrease partitions
- Wide transformation — always shuffles
- Best for rebalancing before a join or aggregation

coalesce()

- Mainly used to reduce partitions
- Narrow transformation — no shuffle
- Cheaper when you only need fewer partitions

Spark builds a plan, optimizes it, then executes — it never runs your code immediately.

Stage	What Happens	Who Performs It
Unresolved Logical Plan	Parses the query; tables/columns not yet verified	SQL Parser
Resolved (Analyzed) Logical Plan	Verifies tables/columns exist, resolves types, checks semantic errors	Catalyst Analyzer
Optimized Logical Plan	Predicate pushdown, column pruning, constant folding, join optimization	Catalyst Optimizer
Physical Plan	Chooses the actual execution strategy — scans, join algorithms — then executes	Physical Planner

INTERVIEW QUESTION BANK

50+ Practice Questions

Organized into 8 categories — from fundamentals to deployment — going beyond the core tutorial content above.



Spark Fundamentals & Ecosystem

INTERVIEW Q&A BANK

Q1. Why is Spark preferred over MapReduce for iterative algorithms?

MapReduce writes intermediate results to disk after every step, which is slow for iterative workloads like machine learning. Spark keeps intermediate data in memory across iterations, avoiding repeated disk I/O and making iterative jobs dramatically faster.

Q2. Is Spark a replacement for Hadoop?

No. Spark replaces Hadoop's MapReduce processing engine, not Hadoop itself. Spark still commonly relies on HDFS for storage and YARN for cluster management — it's a processing engine that can run on top of Hadoop, standalone, or on Kubernetes.



Spark Fundamentals & Ecosystem

INTERVIEW Q&A BANK

Q3. What are the main components of the Spark ecosystem?

Spark Core (the engine), Spark SQL (structured data + Catalyst), Spark Structured Streaming (real-time processing), MLlib (machine learning), and GraphX (graph processing) — all built on the same core engine so they interoperate on the same DataFrame/RDD abstractions.

Q4. What is the difference between client and cluster deploy modes?

In client mode, the driver runs on the machine that submitted the job — useful for interactive work, but the driver dies if that machine disconnects. In cluster mode, the driver runs inside the cluster on a worker node, making it more resilient for production/scheduled jobs.



RDD vs DataFrame vs Dataset

Q5. What's the difference between RDD, DataFrame, and Dataset?

RDD is the low-level, schema-less, immutable distributed collection — full control but no Catalyst optimization. DataFrame is organized into named columns with a schema, benefiting from Catalyst and Tungsten. Dataset (Scala/Java only) adds compile-time type safety; PySpark's DataFrame is effectively Dataset[Row].

Q6. When would you still use RDDs instead of DataFrames?

When you need fine-grained control over partitioning/physical execution, are working with unstructured data that doesn't fit a schema, or need custom low-level transformations the DataFrame API doesn't expose.



RDD vs DataFrame vs Dataset

INTERVIEW Q&A BANK

Q7. What is Tungsten in Spark?

Tungsten is Spark's execution engine optimization initiative focused on CPU and memory efficiency — off-heap binary memory management, whole-stage code generation, and cache-aware computation — working alongside Catalyst to speed up DataFrame/Dataset execution.



Transformations, Actions & Lazy Evaluation

INTERVIEW Q&A BANK

Q8. Why does Spark use lazy evaluation instead of executing each line immediately?

Laziness lets Catalyst see the entire chain of transformations before running anything, so it can reorder operations, prune unused columns, combine steps, and avoid unnecessary computation — producing a globally optimized plan.

Q9. Give an example of an operation that is sometimes narrow and sometimes wide.

`groupBy()` followed by an aggregation is typically wide (requires a shuffle). However, if data is already partitioned/co-located by the grouping key, Spark can sometimes avoid a full shuffle for the aggregation.



Transformations, Actions & Lazy Evaluation

INTERVIEW Q&A BANK

Q10. What's the difference between `cache()` and `persist()`?

`cache()` is shorthand for `persist()` with the default storage level (`MEMORY_AND_DISK` in recent Spark). `persist()` lets you explicitly choose the storage level — e.g. `MEMORY_ONLY`, `DISK_ONLY`, or serialized variants.

Q11. Why should you call `.cache()` before reusing a `DataFrame` multiple times?

Without caching, Spark recomputes the entire lineage from source every time an action is called. Caching materializes the result once so subsequent actions reuse it instead of re-reading and re-transforming from scratch.



Joins & Shuffling

Q12. What is a broadcast join and when should you use it?

A broadcast join sends a small DataFrame in full to every executor, avoiding a shuffle of the large table entirely. Ideal when one side fits in executor memory (`spark.sql.autoBroadcastJoinThreshold` defaults to 10MB).

Q13. What is a Sort-Merge Join?

Spark's default strategy for joining two large DataFrames: both sides are shuffled so matching keys land on the same partition, each partition is sorted by the join key, and matching rows are merged.



Joins & Shuffling

Q14. What is data skew and how do you handle it?

Data skew happens when certain keys have disproportionately more rows, causing straggler tasks. Fixes: salting the skewed key, broadcasting the smaller side of the join, or using Adaptive Query Execution (AQE).

Q15. What is Adaptive Query Execution (AQE)?

A Spark SQL feature that re-optimizes the query plan at runtime using actual shuffle statistics — it can coalesce shuffle partitions, switch to a broadcast join, and handle skewed joins automatically.



Partitioning & Performance Tuning

INTERVIEW Q&A BANK

Q16. How do you decide the right number of partitions?

A common rule of thumb is 2-3 tasks per CPU core across the cluster, with partition sizes around 100-200MB. Too few partitions under-utilizes the cluster; too many creates scheduling overhead from tiny tasks.

Q17. What is `spark.sql.shuffle.partitions` and why does its default matter?

Controls how many partitions Spark uses after a shuffle in SQL/DataFrame operations — default is 200. Too many for small data creates overhead; too few for large data limits parallelism. AQE can auto-tune this.



Partitioning & Performance Tuning

INTERVIEW Q&A BANK

Q18. What is predicate pushdown?

An optimization where Catalyst pushes filter conditions as close to the data source as possible, so only matching rows are actually read from disk — reducing I/O rather than reading everything and filtering in memory.

Q19. Why is Parquet generally preferred over CSV for large-scale Spark workloads?

Parquet is columnar, compressed, and has an embedded schema and statistics, enabling column pruning and predicate pushdown. CSV is row-based, uncompressed, and schema-less — Spark must infer types and read entire rows.



Partitioning & Performance Tuning

INTERVIEW Q&A BANK

Q20. `repartition(n, col)` vs `write.partitionBy(col)` — what's the difference?

`repartition(col)` redistributes data across in-memory partitions for the current job. `write.partitionBy(col)` writes separate folders per distinct value on disk, letting future reads skip entire folders when filtering (partition pruning).



Fault Tolerance & Execution Model

INTERVIEW Q&A BANK

Q21. How does Spark recover from a lost executor or partition?

Spark tracks lineage — the DAG of transformations — for every RDD/DataFrame. If a partition is lost, Spark recomputes just that partition from its original source or last checkpoint, without restarting the whole job.

Q22. What is checkpointing, and how is it different from caching?

Checkpointing writes data to reliable storage and truncates lineage, so recovery doesn't replay a long chain. Caching keeps data in memory/disk but keeps the full lineage — a lost cached partition is recomputed, a checkpointed one is re-read.



Fault Tolerance & Execution Model

INTERVIEW Q&A BANK

Q23. What happens if the driver fails?

The driver holds the SparkContext and application state, so if it fails the whole application typically fails — unless running in cluster mode with supervision, where the cluster manager can restart it. Executor failures, by contrast, Spark recovers automatically via lineage.



Spark SQL & Catalyst Internals

INTERVIEW Q&A BANK

Q24. Walk through what happens when you run `df.explain(True)`.

It prints four plans in sequence: the parsed/unresolved logical plan, the analyzed logical plan (columns/types validated), the optimized logical plan (Catalyst rules applied), and the final physical plan (concrete join/scan strategies).

Q25. What is column pruning?

An optimization where Catalyst removes columns that are never referenced downstream, so Spark only reads the columns it actually needs — especially impactful with columnar formats like Parquet.



Q26. Why can a user-defined function (UDF) hurt performance vs built-in functions?

Built-in functions run inside the JVM/Tungsten engine and are visible to Catalyst, which can optimize around them. Python UDFs require serializing data row-by-row out to a Python process (unless vectorized), adding overhead and blocking optimizations.

Q27. What is a Pandas UDF (vectorized UDF) and why is it faster?

A Pandas UDF operates on batches of rows as Pandas Series/DataFrames using Apache Arrow for efficient data transfer, instead of row-by-row serialization — significantly reducing overhead vs a traditional Python UDF.



Deployment & Cluster Managers

INTERVIEW Q&A BANK

Q28. What cluster managers can Spark run on?

Spark's own Standalone manager, Hadoop YARN, Kubernetes, and historically Apache Mesos. The cluster manager allocates resources — deciding which nodes have available CPU/memory — while Spark schedules tasks within those resources.

Q29. --num-executors vs --executor-cores vs --executor-memory?

--num-executors sets how many executor processes launch, --executor-cores sets cores (and concurrent tasks) per executor, and --executor-memory sets the JVM heap per executor. Together they determine total parallelism and memory budget.



Deployment & Cluster Managers

INTERVIEW Q&A BANK

Q30. What is dynamic allocation in Spark?

A feature that scales the number of executors up or down automatically during a job based on workload, instead of a fixed number set at submit time — useful for shared clusters with uneven resource needs over a job's lifetime.



Rapid-Fire / Conceptual Traps

Q31. True or false: calling `.filter()` immediately scans and filters the data.

False. `.filter()` is a transformation — it only builds up the logical plan. Nothing runs until an action like `.show()` or `.count()` is called.

Q32. Does `repartition()` always cause a shuffle, even to reduce partitions?

Yes. `repartition()` is always a wide transformation that triggers a full shuffle, regardless of direction. To reduce partitions without a shuffle, use `coalesce()` instead.

Q33. How many Spark jobs does `df.count()`; `df.show()` trigger?

Two jobs — each action triggers its own job, and by default Spark recomputes the DataFrame's transformations from scratch for each one unless it was cached beforehand.



Rapid-Fire / Conceptual Traps

INTERVIEW Q&A BANK

Q34. Is an RDD mutable once created?

No. RDDs are immutable — any transformation produces a brand-new RDD rather than modifying the original. This immutability is central to how Spark achieves fault tolerance through lineage.

Q35. Does a narrow transformation ever start a new stage?

No. Narrow transformations (map, filter, union, etc.) never require a shuffle, so they're pipelined into the current stage. Only a wide transformation creates a new stage boundary.

PART TWO



PySpark Transformations — Foundations

Reading data, defining schemas, and the core row/column transformations you'll use in almost every pipeline.



- PySpark is the heart of Apache Spark
- PySpark is the Python API for Apache Spark — it enables real-time, large-scale data processing in a distributed environment using Python
- PySpark supports all of Spark's features: Spark SQL, DataFrames, Structured Streaming, MLlib, pipelines, and Spark Core
- The Pandas API is enabled on top of Spark

Practice Environment

- Commonly practiced and run on Databricks

header=True treats row 1 as column names · inferSchema=True auto-detects data types · load refers to the dataset path.

```
# CSV
df = spark.read.format("csv") \
    .option("header", True) \
    .option("inferSchema", True) \
    .load("/path/gold.dim_customers.csv")

# JSON
df = spark.read.format("json") \
    .option("multiline", True).load("data.json")

# Parquet
df = spark.read.format("parquet").load("customers.parquet")

# JDBC (MySQL / Postgres / SQL Server / Oracle)
df = spark.read.jdbc(url=my_url, table="orders", properties=conn)

df.show()           # first 20 rows
df.printSchema()    # column names, types, nullability
```

Deciding what Spark does when data doesn't match the expected schema (missing fields, invalid types, corruption).

Mode	What Happens	Use Case
PERMISSIVE (default)	Invalid fields become null; processing continues	Most production ETL pipelines
DROPMALFORMED	Malformed rows are discarded entirely	When bad records can be safely ignored
FAILFAST	Job fails on the very first malformed record	Strict data validation and quality enforcement
badRecordsPath	Saves malformed records to a separate location	Auditing and debugging data quality issues

Define Schema for Raw Data

- Explicitly specifying column names, types, and nullability before reading — instead of letting Spark infer them
- **Faster** — Spark doesn't need to scan the data to guess types
- **Better quality** — ensures expected types, detects invalid records early
- **Consistent** — the schema stays the same on every read

```
from pyspark.sql.types import *

schema = StructType([
    StructField("customer_id", IntegerType(), False),
    StructField("name", StringType(), True),
    StructField("age", IntegerType(), True),
    StructField("salary", DoubleType(), True)
])

df = spark.read.schema(schema) \
    .option("header", True).csv("customers.csv")
```


select() chooses specific columns; it can also rename columns, create new ones from expressions, and run calculations. An alias is a temporary name for the current query only.

```
# Select one or more columns
```

```
df.select("name", "salary").show()
```

```
# Select all columns
```

```
df.select("*").show()
```

```
# Alias – temporary name for a column
```

```
df.select(col("salary").alias("employee_salary")).show()
```

Keeps only the rows that satisfy a condition. It's a lazy, narrow transformation — no shuffle, and nothing executes until an action is called.

```
df.filter(df.age > 30).show()
df.filter(col("salary").isNull()).show()
df.filter((df.age < 30) | (df.salary > 7500)).show()
df.filter((df.age > 30) & (df.salary >= 7000)).show()
df.filter(col("department").isin("IT", "HR")).show()
```

withColumn(), withColumnRenamed() & lit()

withColumn() adds or replaces a column · withColumnRenamed() renames a column · lit() creates a column holding a constant value for every row. All are lazy transformations.

```
# withColumn – add / replace a column
df2 = df.withColumn("bonus", col("salary") * 0.10)
df2 = df.withColumn("salary", col("salary").cast("double"))

# withColumnRenamed – rename a column
df2 = df.withColumnRenamed("fname", "first_name")

# lit – constant value for every row
df2 = df.withColumn("country", lit("Egypt"))
df2 = df.withColumn("is_active", lit(True))
```

regexp_replace() edits text with a regex · concat_ws() joins strings with a separator and skips nulls · round() rounds numeric values.

```
# Remove all hyphens from a phone number
```

```
df2 = df.withColumn("phone", regexp_replace(col("phone"), "-", ""))
```

```
# Join first + last name with a space, skipping nulls
```

```
df2 = df.withColumn("full_name", concat_ws(" ", "first_name", "last_name"))
```

```
# Round to 2 decimal places
```

```
df2 = df.withColumn("price", round(col("price"), 2))
```

Converting a column from one data type to another — common after reading from CSV/JSON, where columns default to StringType.

Data Type	Description
"string"	Text
"int" / "integer"	Integer
"long"	Large integer
"double" / "float"	Decimal / floating-point number
"boolean"	True or False
"date" / "timestamp"	Date, or date and time
"decimal(10,2)"	Fixed-precision decimal

sort() and orderBy() are equivalent — both sort ascending or descending on one or more columns.

```
df.sort("salary").show()           # ascending
df.sort(col("salary").desc()).show() # descending
df.sort("salary", "name").show()    # multi-column
df.sort(col("salary").desc(), col("name").asc()).show() # mixed
df.orderBy("salary").show()         # same as sort()
```

What It Does

- Returns the first N rows of a DataFrame
- Commonly used to preview a dataset or test queries on a small sample

Examples

- `df.limit(3).show()` — first 3 rows
- `df.orderBy(col("salary").desc()).limit(3).show()` — top 3 salaries

drop() & dropDuplicates()

- **drop()** — removes one or more columns (not rows)
- **dropDuplicates()** — removes duplicate rows, keeping one occurrence
- **subset** — removes duplicates based on specific columns only
- **Note** — with a subset, PySpark keeps an arbitrary row per unique value; use a window function to keep the latest/highest instead

```
# Drop columns
df2 = df.drop("age")
df2 = df.drop("id", "age")

# Remove all duplicate rows
df2 = df.dropDuplicates()

# Duplicates on a subset of columns
df2 = df.dropDuplicates(["id"])
```


union() & unionByName()

union()

- Combines rows by column position
- Column count, order & types must match
- Doesn't remove duplicates — use distinct()

unionByName()

- Combines rows by column name
- Column order doesn't matter
- Names & types must still be compatible

Function	Description
current_date() / current_timestamp()	Current date / current date and time
to_date() / to_timestamp()	Convert a string to DateType / TimestampType
date_format()	Format a date or timestamp
datediff() / months_between()	Difference between two dates, in days / months
date_add() / date_sub() / add_months()	Add or subtract days / months
year(), month(), dayofmonth(), dayofweek()	Extract individual date parts
last_day() / next_day()	Last day of the month / next occurrence of a weekday

PART THREE



PySpark Transformations — Advanced

String and numeric functions, arrays, grouping and pivoting, joins, window functions, UDFs, and writing data — including Delta.

PySpark's built-in text-cleaning toolkit: case & whitespace, combining & extracting, pattern matching, and boolean checks.

Case & whitespace

```
df.select(upper("name"), lower("name"), initcap("name"), trim("name"))
```

Combining & extracting

```
df.select(concat_ws(" ", "first_name", "last_name").alias("full_name"))
```

```
df.select(substring("name", 1, 4), split("email", "@"))
```

Pattern matching & replacement

```
df.select(regex_replace("phone", "-", ""), instr("email", "@"))
```

Boolean checks

```
df.filter(col("email").contains("gmail"))
```

```
df.filter(col("name").like("J%"))
```

- **isNull()** / **isNotNull()** — detect null / non-null values
- **dropna()** — remove rows with nulls (whole row, a subset, or only if all null)
- **fillna()** — replace nulls with a default value, per-column if needed
- **coalesce()** — returns the first non-null value across columns
- **Best practice** — **dropna()** when missing rows aren't useful, **fillna()** to preserve rows, **coalesce()** to combine columns

```
df2 = df.dropna(subset=["salary"])  
df2 = df.fillna({"salary": 0, "city": "Unknown"})  
df.select(coalesce("phone", "mobile").alias("contact"))
```

3

Numeric Functions

Function	Purpose
round() / bround()	Round to N decimals (bround = round half to even)
abs()	Absolute value
ceil() / floor()	Round up / down to the nearest integer
greatest() / least()	Largest / smallest value across multiple columns
sqrt() / pow()	Square root / raise to a power
rand()	Generate random values (testing / sample data)

split() divides a string into an array on a pattern; square-bracket indexing accesses a specific element.

```
df2 = df.withColumn("name_array", split("full_name", " "))
```

```
# Access array elements
```

```
df2 = (df
    .withColumn("first_name", split("full_name", " ")[0])
    .withColumn("last_name", split("full_name", " ")[1]))
```

```
# Extract username & domain from an email
```

```
df2 = (df
    .withColumn("username", split("email", "@")[0])
    .withColumn("domain", split("email", "@")[1]))
```

Arrays: explode() & array_contains()

- **explode()** — converts each array element (or map entry) into its own row — one row per value
- **array_contains()** — checks whether an array column contains a value; returns a Boolean
- **Common uses** — check if a customer bought a product, find users with a skill, filter on values inside an array
- **Gotcha** — array_contains() only works on array columns — split() a delimited string first

```
df.select("employee_id",  
explode("skills").alias("skill"))
```

```
df.select("name", array_contains("skills", "Python"))  
df.filter(array_contains("skills", "Python")).show()
```


groupBy() groups rows sharing a value, usually followed by *count()*, *sum()*, *avg()*, *min()*, or *max()*.

```
df.groupBy("department").agg(  
    count("*").alias("employees"),  
    sum("salary").alias("total_salary"),  
    avg("salary").alias("avg_salary"),  
    max("salary").alias("highest_salary")  
)  
.show()  
  
# count() vs countDistinct() vs approx_count_distinct()  
df.select(count("city"), countDistinct("city"))
```

collect_list()

- Collects all values from a column into an array
- Keeps duplicate values
- Order is not guaranteed

collect_set()

- Collects the unique values into an array
- Removes duplicates
- Order is not guaranteed

8 pivot()

Transforms row values into columns — like an Excel pivot table. Read it as: `groupBy()` defines the rows, `pivot()` creates new columns from distinct values, `agg()` calculates each cell.

```
df.groupBy("region") \
  .pivot("product") \
  .agg(sum("sales")) \
  .show()

# Employees hired per year, per department
customers.withColumn("year", year("hire_date")) \
  .groupBy("year").pivot("department").agg(count("*")).show()
```

Conditional columns — like IF...ELSE or SQL's CASE WHEN. Chain multiple when() calls before a single otherwise(); unmatched rows get NULL if otherwise() is omitted.

```
df.withColumn("salary_level",  
    when(df.salary >= 7000, "High").otherwise("Low"))  
  
df.withColumn("category",  
    when(df.age < 18, "Child")  
    .when(df.age < 60, "Adult")  
    .otherwise("Senior"))
```

10 Joins

df1.join(df2, join_condition, join_type) — combines rows from two DataFrames on a common column or condition.

Join Type	Keeps
inner	Matching rows only
left / right	All rows from that side + matching rows from the other
outer (full)	All rows from both DataFrames
left_semi	Only matching rows from the left DataFrame (no right columns)
left_anti	Only non-matching rows from the left DataFrame
cross	Every combination of rows

Calculate across a group of rows without collapsing them. Unlike `groupBy()`, every original row is kept — need one row per group? use `groupBy()`. Need to keep every row? use a window function.

```
from pyspark.sql.window import Window

w = Window.partitionBy("department").orderBy("salary")

df.withColumn("rank", rank().over(w))
df.withColumn("prev_salary", lag("salary", 1).over(w))

# Running total (cumulative sum)
w2 = Window.orderBy("employee_id") \
    .rowsBetween(Window.unboundedPreceding, Window.currentRow)
df.withColumn("cumulative_salary", sum("salary").over(w2))
```

User-Defined Function (UDF)

- Applies your own Python function to a column
- Returns one value per row
- Prefer built-in functions when possible — they're Catalyst-optimized and faster

User-Defined Table Function (UDTF)

- Returns multiple rows and/or columns per input
- Use to split text into rows, expand arrays, or flatten nested JSON
- Example: skills "Python,SQL,Spark" -> one row per skill

df.write.mode("overwrite").parquet("output/employees") — the write mode works the same across CSV, Parquet, JSON, ORC, Hive tables, and JDBC.

Mode	Behavior
append	Adds new data; keeps old data and adds new rows
overwrite	Replaces existing data — deletes old data first
ignore	Does nothing if output already exists; old data is kept
error (default)	Throws an exception if the output already exists

- Delta Lake is an open table format built on top of Parquet
- A Delta table = Parquet files (the actual data) + a `_delta_log` folder (transaction logs, metadata, schema, version history)
- The transaction log enables ACID transactions, time travel, UPDATE, DELETE, MERGE, and schema enforcement/evolution
- `df.write.format("delta").mode("overwrite").save(path)` · `spark.read.format("delta").load(path)`

Why it matters: Delta is the backbone of modern medallion (bronze / silver / gold) data-lake architectures.

YOU'RE READY.

Hadoop & HDFS · Spark Architecture · RDDs & DataFrames · Catalyst & Tungsten · Joins & Windows · 50+ interview questions